Hochschule Wismar
University of Applied Sciences
Technology, Business and Design
Fakultät für Ingenieurwissenschaften

Mobile Applikation: Hackintosh Companion

Entwicklung einer mobilen Begleit-Applikation zur datenbankgestützten Verwaltung von Hackintosh-Systemen mit Flutter und SQLite

Eingereicht am: 11.04.2026

von: Kilian Ketelhohn (520858)
geboren am 27.11.2001 in Schwerin

Betreuer: Prof. Dr. Matthias Kreuseler

Aufgabenstellung

0.1 Aufgabenstellung

Die Installation und der Betrieb von macOS auf nicht offiziell unterstützter Hardware (Hackintosh) stellt Anwender vor erhebliche Herausforderungen. Der Prozess erfordert die präzise Abstimmung von Hardware-IDs, ACPI-Patches, Kernel-Erweiterungen (Kexts) und Bootloader-Konfigurationen. Ein zentrales Problem in der Community ist die mangelnde Mobilität und Strukturierung dieser Informationen: Während der Konfigurationsphase ist das Zielgerät oft nicht einsatzbereit, und wichtige Daten sind über komplexe Web-Guides verteilt.

0.2 Ziel der Arbeit

Das Ziel dieser Projektarbeit ist die Konzeption und Implementierung der **Hackintosh Companion App**. Hierbei handelt es sich um eine mobile Cross-Plattform-Anwendung auf Basis von **Flutter**, die als zentrale Wissensdatenbank und technisches Werkzeug fungiert. Die Anwendung soll den Nutzer dabei unterstützen, gerätespezifische Konfigurationen strukturiert zu erfassen, zu visualisieren und mit einer Community-Datenbank zu synchronisieren.

0.3 Konkrete Anforderungen und Teilaufgaben

Im Rahmen der Softwareentwicklung sind folgende Teilaufgaben zu lösen:

- **Datenmodellierung:** Entwurf und Implementierung eines relationalen Datenbankmodells in **SQLite**, um komplexe Abhängigkeiten zwischen Hardware-Komponenten und den benötigten Treibern abzubilden.
- **Benutzeroberfläche und Animation:** Entwicklung eines modernen, konsistenten **Dark-Mode-Designs**. Ein besonderer Fokus liegt auf der Implementierung einer **Boot-Animation**, die den Startvorgang eines Hackintosh-Systems (BIOS, OpenCore, macOS-Kernel) simuliert, um die Nutzererfahrung zu steigern.
- **Hardware-Visualisierung:** Integration einer interaktiven **3D-Grafik-Komponente**, die es dem Nutzer ermöglicht, Hardware-Komponenten am Gerät visuell zu identifizieren und Informationen zu deren Kompatibilität abzurufen.

- **Konnektivität und Synchronisation:** Implementierung eines **REST-API-Service** zur Synchronisation lokaler Daten mit einem Server (*Offline-First-Ansatz*) sowie die Einbindung von **QR-Code-Scanning** zur Erfassung externer Konfigurations-Strings.
- **Qualitätssicherung:** Sicherstellung der Performance und Stabilität auf physischer Android-Hardware (Samsung A3 2016) unter Berücksichtigung limitierter Systemressourcen.

0.4 Methodische Grundlage

Als fachliche Referenz für die in der App abgebildeten Logiken dienen die Industriestandards der Hackintosh-Community, insbesondere der *OpenCore Install Guide* von Dortania. Die technische Umsetzung folgt den Best Practices der Flutter-Entwicklung, insbesondere im Bereich der asynchronen Programmierung und des State-Managements.

Inhaltsverzeichnis

0.1 Aufgabenstellung . . . 2
0.2 Ziel der Arbeit . . . 2
0.3 Konkrete Anforderungen und Teilaufgaben . . . 2
0.4 Methodische Grundlage . . . 3

1 Einleitung **6**
1.0.1 Motivation . . . 6
1.0.2 Zielsetzung des Projekts . . . 6
1.0.3 Struktur der Dokumentation . . . 7
1.1 Grundlagen . . . 7
1.2 Das Hackintosh-Ökosystem . . . 7
1.2.1 Technische Säulen: Bootloader und Kexts . . . 7
1.3 Framework: Flutter und Dart . . . 8
1.4 Datenhaltung: SQLite und Offline-First . . . 8
1.4.1 Relationale Datenbank mit SQLite . . . 8
1.4.2 Offline-First-Konzept . . . 8
1.5 Konnektivität: REST-API und JSON . . . 9
1.6 Motivation und Problemstellung . . . 9
1.6.1 Problemstellung . . . 9
1.6.2 Zielsetzung der Arbeit . . . 9
1.7 Zusammenfassung für Fachfremde . . . 9
1.8 Methodik . . . 10
1.9 Vorgehensweise . . . 10
1.10 Konzept des relationalen Datenmodells . . . 10
1.10.1 Entitäts-Relationship-Modell (ERM) . . . 10
1.10.2 Methodische Designentscheidungen . . . 11
1.11 Systemarchitektur und Datenfluss . . . 11
1.12 Zusammenfassung . . . 11
1.13 Praktische Umsetzung: Implementierung . . . 12
1.14 Einleitung . . . 12
1.15 Workflow der Implementierung . . . 12
1.16 Integration der Systemkomponenten . . . 12
1.16.1 Lokale Datenhaltung und Offline-First . . . 12
1.16.2 Schnittstellen-Architektur . . . 13
1.17 Praktische Umsetzungsschritte . . . 13
1.18 Besondere Herausforderungen und Lösungen . . . 13
1.19 Testing und Qualitätssicherung . . . 14
1.20 Zusammenfassung . . . 14
1.21 Evaluation . . . 14

1.22 Testumgebung und Datensätze 14
1.23 Performance-Analyse 14
1.23.1 Datenbank-Performance (SQLite) 14
1.23.2 Grafik- und Animations-Performance 15
1.24 Funktionale Ergebnisse 15
1.25 Zusammenfassende Beurteilung 15
1.26 Fazit und Ausblick 16
1.27 Zusammenfassung der Ergebnisse 16
1.28 Persönliches Resümee 16
1.29 Ausblick 16
.1 Datenbank-Implementierung (SQLite) 17
.2 REST-API Synchronisation 18
.3 3D-Visualisierung 18
.4 Simulation der Boot-Sequenz 19

Abbildungsverzeichnis **20**

Tabellenverzeichnis **21**

Listings **22**

Selbstständigkeitserklärung **23**

1 Einleitung

1.0.1 Motivation

Die Installation des Betriebssystems macOS auf Hardware, die nicht von Apple Inc. stammt – ein Prozess, der in der IT-Community als *Hackintoshing* bekannt ist – stellt eine komplexe technische Herausforderung dar. Besonders bei Notebooks, wie den verbreiteten Modellen der Lenovo ThinkPad-Serie, erfordert die Emulation des Apple-spezifischen EFI-Umfelds mittels Bootloadern wie **OpenCore** eine präzise Abstimmung von Hardware-Informationen, BIOS-Einstellungen und Kernel-Erweiterungen (Kexts).

Häufig müssen Nutzer während der Installation oder bei Systemaktualisierungen schnell auf spezifische Konfigurationsdaten zugreifen, während das Zielgerät selbst nicht einsatzbereit ist. Hier setzt die **Hackintosh Companion App** an: Sie dient als mobiles Begleitwerkzeug und zentrale Wissensdatenbank, um Konfigurationen geräteunabhängig zu verwalten und Hardware-Spezifikationen interaktiv zu visualisieren.

1.0.2 Zielsetzung des Projekts

Das primäre Ziel dieser Arbeit ist die Entwicklung einer funktionalen, visuell ansprechenden Cross-Plattform-Anwendung unter Verwendung des **Flutter-Frameworks**. Die App soll nicht nur als einfache Datenbank fungieren, sondern durch moderne Technologien den Workflow eines Systemadministrators oder Hackintosh-Enthusiasten optimieren.

Zu den Kernzielen gehören:

- **Datenmanagement:** Implementierung einer relationalen Datenbankstruktur mit SQLite zur Speicherung komplexer Hardware-Profile.
- **Synchronisation:** Gewährleistung der Datenverfügbarkeit durch einen Offline-First-Ansatz und REST-API-Integration.

- **Benutzererlebnis (UX):** Einsatz von Computergrafiken und 3D-Modellen zur intuitiven Hardware-Identifikation sowie die Simulation technischer Boot-Prozesse durch Animationen.
- **Sensorintegration:** Nutzung der mobilen Kamera zur Erfassung von Konfigurationsdaten via QR-Code.

1.0.3 Struktur der Dokumentation

Die vorliegende Dokumentation beschreibt den vollständigen Entwicklungsprozess – von der konzeptionellen Planung über das Datenbankdesign und die Implementierung technischer Highlights bis hin zur Bereitstellung der finalen Android-Applikation.

1.1 Grundlagen

1.2 Das Hackintosh-Ökosystem

Ein *Hackintosh* bezeichnet ein Computersystem, auf dem das Apple-Betriebssystem macOS auf Hardware ausgeführt wird, die nicht von Apple offiziell autorisiert wurde. Da Apple Hard- und Software als geschlossenes System konzipiert, erfordert der Betrieb auf Standard-PC-Komponenten tiefgreifende Modifikationen an der Schnittstelle zwischen Firmware und Betriebssystem.

1.2.1 Technische Säulen: Bootloader und Kexts

Damit macOS auf x86-basierter Hardware (wie z.B. Lenovo ThinkPads) startet, sind drei Komponenten essenziell:

- **OpenCore Bootloader:** Der moderne Standard-Bootloader. Er emuliert eine Apple-spezifische EFI-Umgebung und injiziert notwendige Daten in den Kernel-Bootprozess.
- **Kexts (Kernel Extensions):** Da macOS für viele PC-Komponenten (WLAN, Trackpads, Ethernet) keine nativen Treiber besitzt, werden Kernel-Erweiterungen genutzt, um die Hardwarekompatibilität herzustellen.
- **ACPI-Patches:** Diese korrigieren die herstellerspezifischen Tabellen der Hardware (DSDT/SSDT), damit macOS Funktionen wie das Batteriemanagement oder den Ruhezustand korrekt anspricht.

1.3 Framework: Flutter und Dart

Die Realisierung der Companion App erfolgt mit dem Framework **Flutter** von Google.

- **Cross-Plattform:** Flutter ermöglicht die Entwicklung aus einer einzigen Codebasis für Android und iOS.
- **Rendering-Engine:** Anders als native Apps nutzt Flutter eine eigene Engine (Skia/Impeller), was hochperformante Animationen (wie die Boot-Simulation) erlaubt.
- **Dart:** Die zugrundeliegende Programmiersprache ist objektorientiert und ermöglicht durch *Ahead-of-Time* (AOT) Kompilierung eine native Ausführungsgeschwindigkeit.

1.4 Datenhaltung: SQLite und Offline-First

Im Gegensatz zu serverbasierten NoSQL-Systemen nutzt dieses Projekt **SQLite** für die lokale Persistenz.

1.4.1 Relationale Datenbank mit SQLite

SQLite ist eine dateibasierte, relationale Datenbank. Für die Hackintosh-Verwaltung bietet sie den Vorteil der strukturierten Datenhaltung:

- **Struktur:** Feste Tabellen für Geräte, Kexts und Issues gewährleisten Datenintegrität.
- **Relationen:** Über Fremdschlüssel (*Foreign Keys*) wird sichergestellt, dass Treiber (Kexts) eindeutig einem Hardware-Profil zugeordnet sind.

1.4.2 Offline-First-Konzept

In der mobilen Programmierung bezeichnet *Offline-First* einen Ansatz, bei dem die App primär mit der lokalen Datenbank arbeitet. Dies ist für Hackintosh-Nutzer kritisch, da während der Systemkonfiguration oft keine stabile Internetverbindung besteht.

1.5 Konnektivität: REST-API und JSON

Zur Synchronisation der lokalen Daten mit einer zentralen Community-Datenbank wird eine **REST-API** (Representational State Transfer) eingesetzt.

- **HTTP-Methoden:** Die Kommunikation erfolgt über Standard-Methoden wie `GET` (Abrufen) und `POST` (Hochladen).
- **JSON:** Als Austauschformat dient *JavaScript Object Notation*. Es ist leichtgewichtig und lässt sich in Dart effizient in Objekte (*Mapping*) umwandeln.

1.6 Motivation und Problemstellung

1.6.1 Problemstellung

Die Konfiguration eines Hackintoshs ist ein iterativer Prozess (TTrial-and-Error"). Informationen über funktionierende Kext-Kombinationen oder BIOS-Einstellungen sind oft über diverse Foren und GitHub-Repositories verstreut. Es fehlt an einer mobilen, zentralisierten Lösung, die diese Daten auch offline am Einsatzort bereitstellt.

1.6.2 Zielsetzung der Arbeit

Das Ziel ist die Entwicklung der **Hackintosh Companion App**, die:

- Eine strukturierte Erfassung von Hardware-Setups ermöglicht.
- Durch **QR-Code-Scanning** den schnellen Austausch von Konfigurationen erlaubt.
- Mittels **3D-Visualisierung** die Identifikation von Hardware-Komponenten vereinfacht.
- Eine Brücke zwischen lokaler Speicherung und globalem Cloud-Sync schlägt.

1.7 Zusammenfassung für Fachfremde

Ein Hackintosh macht macOS auf normalen Laptops nutzbar, was technisch sehr komplex ist. Die hier entwickelte App fungiert als digitales Werkzeugheft. Sie speichert komplizierte Treibereinstellungen lokal ab, visualisiert die Bauteile in 3D und ermöglicht es, fertige Setups einfach per QR-Code mit anderen Nutzern zu teilen – ähnlich wie ein interaktives Handbuch für Systemadministratoren.

1.8 Methodik

1.9 Vorgehensweise

Die methodische Entwicklung der *Hackintosh Companion App* folgt einem strukturierten, ingenieurwissenschaftlichen Ansatz für mobile Systeme. Ziel war es, die oft unübersichtlichen und volatilen Hackintosh-Informationen in eine stabile, mobil verfügbare Form zu überführen.

Die Vorgehensweise gliedert sich in folgende Phasen:

1. **Anforderungsanalyse und Use-Case-Definition:** Identifikation der Bedürfnisse von Hackintosh-Usern. Hierbei wurde festgestellt, dass der Zugriff auf BIOS-Settings und Kext-Listen besonders in Phasen kritisch ist, in denen das Zielsystem (z. B. ein ThinkPad) aufgrund von Fehlkonfigurationen nicht über einen Internetzugang verfügt.
2. **Architekturentwurf (Offline-First):** Methodische Entscheidung für eine hybride Datenhaltung. Die Wahl fiel auf **SQLite** für die lokale Persistenz und eine **REST-API** für den globalen Datenaustausch. Dies stellt die Verfügbarkeit der Daten unter Extrembedingungen sicher.
3. **Prototyping und UI/UX-Design:** Entwicklung eines modernen Dark-Tech-Interfaces in Flutter. Die Methodik sah hierbei vor, komplexe technische Daten durch Animationen (Boot-Sequenz) und interaktive Grafik-Elemente (3D-Modell) intuitiv begreifbar zu machen.

1.10 Konzept des relationalen Datenmodells

Im Gegensatz zu einem schema-losen Ansatz wurde hier ein **relationales Modell** gewählt, um die strikten Abhängigkeiten zwischen Hardwarekomponenten und den dafür notwendigen Treibern (Kexts) technisch abzusichern.

1.10.1 Entitäts-Relationship-Modell (ERM)

Das Modell ist so normalisiert, dass Redundanzen vermieden und die Datenintegrität gewahrt bleibt.

Tabelle	Attribute (Auszug)	Relation
`devices`	id, name, cpu, gpu, compatible	Parent
`kexts`	id, device_id, name, version	1:n zu devices
`issues`	id, device_id, problem, loesung	1:n zu devices
`configs`	id, device_id, bios_settings, oc_version	1:1 zu devices

Tabelle 1: Struktur des relationalen SQLite-Modells

1.10.2 Methodische Designentscheidungen

- **Datenintegrität:** Durch den Einsatz von *Foreign Keys* in SQLite wird methodisch verhindert, dass "verwaisteTTreibereinträge entstehen, wenn ein Gerät gelöscht wird (`ON DELETE CASCADE`).
- **Effizienz durch Abstraktion:** Die Nutzung von **Modell-Klassen** in Dart ermöglicht ein automatisiertes Mapping zwischen Datenbank-JSON und UI-Widgets.
- **Interaktivität:** Die Entscheidung für einen **QR-Code-Workflow** basiert auf der Methodik, manuelle Eingabefehler bei komplexen Konfigurations-Strings zu eliminieren.

1.11 Systemarchitektur und Datenfluss

Methodisch wurde eine Schichtenarchitektur (Layered Architecture) implementiert:

1. **Presentation Layer:** Flutter Widgets (UI).
2. **Business Logic Layer:** Services für Sync und QR-Verarbeitung.
3. **Data Layer:** SQLite Datenbank und REST-Client.

1.12 Zusammenfassung

Die gewählte Methodik kombiniert klassische Datenbank-Prinzipien mit modernen Ansätzen der mobilen App-Entwicklung. Durch die bewusste Entscheidung für ein relationales Modell in Verbindung mit einer Offline-First-Strategie wird eine wissenschaftlich fundierte und zugleich hochgradig praxistaugliche Lösung für die Hackintosh-Community geschaffen.

1.13 Praktische Umsetzung: Implementierung

1.14 Einleitung

In diesem Kapitel wird die technische Realisierung der *Hackintosh Companion App* detailliert beschrieben. Der Fokus liegt auf der Integration der mobilen Benutzeroberfläche mit der lokalen SQLite-Datenbank sowie der Anbindung externer Schnittstellen (REST-API, Sensoren). Es wird aufgezeigt, wie die theoretischen Anforderungen des Konzepts in eine funktionale Android-Applikation überführt wurden.

1.15 Workflow der Implementierung

Die Entwicklung erfolgte nach einem iterativen Vorgehensmodell in folgenden Phasen:

1. **Datenbank-Setup:** Implementierung des `DatabaseHelper`-Singletons zur Steuerung der SQLite-Instanz und Definition des relationalen Schemas (Geräte, Kexts, Issues).
2. **Core-UI & Navigation:** Aufbau der Grundstruktur mit Flutter (Material Design 3). Implementierung der Hero-Animationen für nahtlose Übergänge zwischen Home- und Detail-Screen.
3. **Grafik-Integration:** Entwicklung der zeitgesteuerten Boot-Sequenz (Simulation von BIOS, OpenCore und macOS-Kernel) und Einbindung des 3D-Controllers für die Hardware-Visualisierung.
4. **Sensorik & Cloud-Anbindung:** Integration des `mobile_scanner`-Pakets für die Kamera-Interaktion sowie des `http`-Service für den Datenaustausch mit der REST-API.
5. **Optimierung für Legacy-Hardware:** Anpassung des Renderers in der `AndroidManifest.xml` um die Stabilität auf älteren Grafik-Chips (Mali-GPUs) zu gewährleisten.

1.16 Integration der Systemkomponenten

1.16.1 Lokale Datenhaltung und Offline-First

Die App agiert unabhängig von einer permanenten Internetverbindung. Die Geschäftslogik prüft bei jedem Start den Zustand der lokalen SQLite-Datenbank. Änderungen (z.B. neue Kext-Einträge) werden unmittelbar persistent gespeichert, was die Zuverlässigkeit während eines Hackintosh-Builds sicherstellt.

1.16.2 Schnittstellen-Architektur

Die Kommunikation zwischen den Komponenten erfolgt streng modular:

- **UI zu DB:** Über asynchrone `Future`-Methoden werden Datenströme direkt in die Widgets (mittels `ListView.builder`) injiziert.
- **UI zu Sensoren:** Der QR-Scanner liefert Rohdaten an den `SyncService`, welcher die Informationen validiert und in Geräte-Modelle parst.
- **Cloud-Sync:** Der `SyncService` übernimmt die Konvertierung von Dart-Objekten in JSON-Strings für den API-Upload.

1.17 Praktische Umsetzungsschritte

Die konkrete Programmierung umfasste unter anderem:

- Erstellung der relationalen Tabellen mit `FOREIGN KEY`-Constraints zur Sicherung der Datenintegrität.
- Implementierung eines **Custom Splash Screens** zur Darstellung des Hackintosh-Bootvorgangs mittels `StatefulWidgets` und `Future.delayed`.
- Integration einer **.glb-Datei** (3D-Laptop-Modell), welche interaktive Rotationen und Komponenten-Fokus ermöglicht.
- Aufbau eines `ThemeData`-Systems für einen konsistenten Dark-Tech-Look im gesamten Interface.

1.18 Besondere Herausforderungen und Lösungen

Während der Implementierung traten spezifische Herausforderungen auf:

- **Grafik-Inkompatibilität:** Abstürze durch den neuen Flutter-Renderer *Impeller* auf älteren Android-Geräten wurden durch ein Fallback auf den *Skia*-Renderer gelöst.
- **Daten-Synchronisation:** Die Handhabung von asynchronen API-Aufrufen wurde durch robuste `try-catch`-Blöcke und SnackBar-Feedback für den Nutzer abgesichert.
- **Animation-Performance:** Um Ruckler auf dem Testgerät (Samsung A3) zu vermeiden, wurden komplexe 3D-Operationen und UI-Updates optimiert.

1.19 Testing und Qualitätssicherung

Die Qualitätssicherung erfolgte durch:

- **Gerätetests:** Kontinuierliches Deployment auf physischer Hardware (Samsung A3, Android 13/Custom ROM), um reale Performance-Werte zu erhalten.
- **Validierung:** Prüfung der QR-Code-Logik mit verschiedenen Datensätzen (valide JSON vs. korrupte Daten).
- **UI-Inspektion:** Nutzung des Flutter DevTools-Inspektors zur Optimierung der Widget-Trees und zur Vermeidung von Memory Leaks.

1.20 Zusammenfassung

Durch die Verzahnung von lokaler Datenbank-Expertise, moderner Cross-Plattform-Entwicklung und der gezielten Nutzung von Hardware-Sensoren wurde ein robustes System geschaffen. Die Trennung von Datenlogik (Service-Layer) und Darstellung (View-Layer) ermöglicht eine einfache Skalierbarkeit für zukünftige Community-Features.

1.21 Evaluation

1.22 Testumgebung und Datensätze

Die Evaluation der *Hackintosh Companion App* erfolgte primär auf einem physischen Testgerät des Typs **Samsung Galaxy A3 (2016)** unter Android 13. Zur Überprüfung der Skalierbarkeit und Performance wurden 50 Test-Datensätze generiert, die komplexe Hardware-Konfigurationen (inkl. Listen von Kexts und ACPI-Patches) abbilden.

1.23 Performance-Analyse

1.23.1 Datenbank-Performance (SQLite)

Die lokale Datenhaltung mittels SQLite zeigt eine hohe Effizienz. Lesezugriffe auf die Gerätedatenbank sowie die Filterung nach spezifischen Hardware-IDs erfolgen in weniger als 15ms. Die Implementierung von Indizes auf den Feldern `name` und

`device_id` gewährleistet auch bei steigender Datenmenge eine flüssige Darstellung in der `ListView`.

1.23.2 Grafik- und Animations-Performance

Ein kritischer Punkt war die Ausführung der 3D-Visualisierung und der Boot-Animation auf der limitierten Hardware des Samsung A3.

- **Boot-Animation:** Durch die Nutzung asynchroner Timer (`Future.delayed`) wird der Main-Thread entlastet, was zu einer ruckelfreien Darstellung der BIOS-Logs führt.
- **3D-Rendering:** Die Deaktivierung des ressourcenintensiven *Impeller*-Renderers zugunsten von *Skia* resultierte in einer stabilen Framerate von ca. 50–60 FPS während der Interaktion mit dem Laptop-Modell.

1.24 Funktionale Ergebnisse

Die Evaluation der Kernfunktionen lieferte folgende Ergebnisse:

- **Offline-Verfügbarkeit:** Der *Offline-First-Ansatz* wurde erfolgreich verifiziert. Alle Gerätedaten sind ohne aktive Internetverbindung sofort nach dem App-Start verfügbar.
- **QR-Code-Erkennung:** Der `mobile_scanner` erkennt komplexe JSON-Strings innerhalb von 1,2 Sekunden unter normalen Lichtbedingungen.
- **Cloud-Synchronisation:** Der Datenaustausch via REST-API funktioniert stabil. Die Fehlerbehandlung (*Error Handling*) fängt Verbindungsabbrüche ohne Absturz der Anwendung ab.

1.25 Zusammenfassende Beurteilung

Die App erfüllt alle im Konzept definierten Anforderungen. Besonders hervorzuheben ist die Kombination aus technischer Informationstiefe und intuitiver Benutzerführung. Die Entscheidung für Flutter als Framework ermöglichte es, eine visuell ansprechende Applikation zu entwickeln, die trotz der hardwarenahen Features (3D, Kamera) auch auf älteren Endgeräten eine hohe User Experience bietet.

1.26 Fazit und Ausblick

Die vorliegende Arbeit demonstriert erfolgreich, wie moderne Cross-Plattform-Frameworks und lokale Datenbanktechnologien genutzt werden können, um komplexe IT-Prozesse wie die Hackintosh-Konfiguration mobil zu unterstützen. Die entwickelte *Hackintosh Companion App* überführt statische und verstreute Informationen in ein interaktives, benutzerzentriertes Werkzeug.

1.27 Zusammenfassung der Ergebnisse

Im Rahmen des Projekts wurden folgende Meilensteine erreicht:

- **Strukturierte Datenhaltung:** Durch den Einsatz von SQLite wurde ein relationales Modell implementiert, das Hardware-Spezifikationen, Treiberabhängigkeiten (Kexts) und Problemlösungen (Issues) konsistent verwaltet.
- **Innovative Benutzeroberfläche:** Die Integration einer Boot-Animation und eines interaktiven 3D-Laptop-Modells hebt die App von rein textbasierten Datenbankanwendungen ab und steigert die User Experience massiv.
- **Praxisnahe Konnektivität:** Die Kombination aus Offline-First-Strategie, REST-API-Synchronisation und QR-Code-Scanning stellt sicher, dass die Anwendung unter realen Einsatzbedingungen – etwa bei fehlender Internetverbindung während einer Systeminstallation – zuverlässig funktioniert.
- **Hardware-Optimierung:** Die erfolgreiche Portierung und Performance-Optimierung auf ein älteres Testgerät (Samsung A3 2016) belegt die Effizienz des Flutter-Frameworks und der gewählten Architektur.

1.28 Persönliches Resümee

Die Entwicklung der App bot tiefe Einblicke in die asynchrone Programmierung mit Dart sowie in die Herausforderungen des 3D-Renderings auf mobilen Endgeräten. Besonders die Problematik der Grafik-Renderer (Skia vs. Impeller) verdeutlichte die Notwendigkeit einer engen Verzahnung von Softwareentwicklung und Hardwarekenntnissen.

1.29 Ausblick

Trotz des erreichten Funktionsumfangs bietet die App Potenzial für zukünftige Erweiterungen:

- **Automatisierte Kext-Updates:** Eine Anbindung an die GitHub-API könnte Nutzer automatisch informieren, wenn neue Versionen ihrer installierten Treiber verfügbar sind.
- **Community-Feature:** Implementierung eines Bewertungssystems für Hardware-Konfigurationen, um die Zuverlässigkeit einzelner Setups durch Nutzer-Feedback zu verifizieren.
- **Erweiterte Sensorik:** Nutzung von Machine Learning (z. B. Google ML Kit), um Hardware-Komponenten direkt über die Kamera per Bilderkennung zu identifizieren, ohne auf QR-Codes angewiesen zu sein.

Abschließend lässt sich festhalten, dass die *Hackintosh Companion App* die gestellten Anforderungen nicht nur erfüllt, sondern durch die gewählte technische Tiefe eine solide Grundlage für eine professionelle Community-Plattform darstellt.

.1 Datenbank-Implementierung (SQLite)

Im folgenden Listing wird die Initialisierung der lokalen Datenbank sowie das relationale Schema mit Fremdschlüssel-Abhängigkeiten dargestellt.

```
// Auszug aus lib/database/database_helper.dart
Future _createDB(Database db, int version) async {
  // Haupttabelle fuer Geraete
  await db.execute('''
    CREATE TABLE devices (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      name TEXT NOT NULL,
      cpu TEXT NOT NULL,
      gpu TEXT NOT NULL,
      compatible INTEGER NOT NULL
    )
  ''');

  // Tabelle fuer Kexts mit Relation zu devices
  await db.execute('''
    CREATE TABLE kexts (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      device_id INTEGER,
      name TEXT NOT NULL,
      version TEXT,
```

```
      FOREIGN KEY (device_id) REFERENCES devices (id) ON
         DELETE CASCADE
    )
  ''');
}
```

Listing 1: DatabaseHelper.dart - Relationales Schema und Initialisierung

.2 REST-API Synchronisation

Dieses Listing zeigt die Implementierung des Offline-First-Ansatzes durch den Abgleich lokaler SQLite-Daten mit einem remote JSON-Endpunkt.

```
// Auszug aus lib/services/sync_service.dart
Future<void> syncUp() async {
  final localDevices = await DatabaseHelper.instance.
     getAllDevices();

  for (var device in localDevices) {
    try {
      await http.post(
        Uri.parse('$baseUrl/devices'),
        body: json.encode(device.toMap()),
        headers: {'Content-Type': 'application/json'},
      );
    } catch (e) {
      debugPrint("Upload-Fehler: $e");
    }
  }
}
```

Listing 2: SyncService.dart - REST-API Anbindung

.3 3D-Visualisierung

Auszug der Widget-Logik zur Einbindung des interaktiven Laptop-Modells im Detail-Screen.

```
// Auszug aus lib/screens/visual_screen.dart
@override
Widget build(BuildContext context) {
  return Flutter3DViewer(
```

```
    controller: controller,
    src: 'assets/models/thinkpad.glb', // Pfad zum 3D-Asset
  );
}
```

Listing 3: VisualScreen.dart - 3D-Modell Einbindung

.4 Simulation der Boot-Sequenz

Implementierung der zeitgesteuerten Phasen (BIOS, OpenCore, macOS) zur Steigerung der User Experience.

```
// Auszug aus lib/screens/boot_animation_screen.dart
void _startBootSequence() async {
  // Phase 1: BIOS Logs
  await Future.delayed(Duration(milliseconds: 300));
  setState(() => _bootPhase = 0);

  // Phase 2: OpenCore Picker
  await Future.delayed(Duration(seconds: 2));
  setState(() => _bootPhase = 1);

  // Phase 3: macOS Start
  await Future.delayed(Duration(seconds: 2));
  setState(() => _bootPhase = 2);
}
```

Listing 4: BootAnimationScreen.dart - Phasensteuerung

Abbildungsverzeichnis

Tabellenverzeichnis

1 Struktur des relationalen SQLite-Modells 11

Listings

1 DatabaseHelper.dart - Relationales Schema und Initialisierung 17
2 SyncService.dart - REST-API Anbindung 18
3 VisualScreen.dart - 3D-Modell Einbindung 18
4 BootAnimationScreen.dart - Phasensteuerung 19

Selbstständigkeitserklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig verfasst und keine anderen als die angegebenen Hilfsmittel benutzt habe. Die Stellen der Arbeit, die anderen Quellen im Wortlaut oder dem Sinn nach entnommen wurden, sind durch Angaben der Herkunft kenntlich gemacht. Dies gilt auch für Zeichnungen, Skizzen, bildliche Darstellungen sowie für Quellen aus dem Internet.

Ich erkläre ferner, dass ich die vorliegende Arbeit in keinem anderen Prüfungsverfahren als Prüfungsarbeit eingereicht habe oder einreichen werde.

Die eingereichte schriftliche Arbeit entspricht der elektronischen Fassung. Ich stimme zu, dass eine elektronische Kopie gefertigt und gespeichert werden darf, um eine Überprüfung mittels Anti-Plagiatssoftware zu ermöglichen.

Ort, Datum

Unterschrift

Thesen

Mobile Applikation: Hackintosh Companion

Entwicklung einer mobilen Begleit-Applikation zur datenbankgestützten Verwaltung von Hackintosh-Systemen mit Flutter und SQLite

Eingereicht am:	11.04.2026
von:	Kilian Ketelhohn (520858) geboren am 27.11.2001 in Schwerin
Betreuer:	Prof. Dr. Matthias Kreuseler

- Die Nutzung von Cross-Plattform-Frameworks (Flutter) ermöglicht eine effiziente Bereitstellung technischer Datenbanken für den mobilen Einsatz
- Ein Offline-First-Ansatz ist essenziell für IT-Begleitwerkzeuge in infrastrukturkritischen Phasen
- Die Simulation technischer Prozesse (Boot-Animation) steigert die User Experience und das Systemverständnis
- Relationale Datenbanken (SQLite) gewährleisten die Integrität komplexer Hardware-Treiber-Beziehungen
- Die Integration von 3D-Grafikkomponenten unterstützt die intuitive Identifikation physischer Hardware-Bauteile
- QR-Code-gestützter Datenaustausch minimiert manuelle Fehlerquellen bei der Konfigurationsübertragung
- Die Optimierung für Legacy-Hardware (Samsung A3) belegt die Praxistauglichkeit moderner mobiler Engines.